

Answers for Week 3

1. Cache Effects

The goal of this exercise is to visualize how the caches affects performance. The exercise will expand on the example from section 6.1.1. in Fast Python. Consider the following code:

```
1 import numpy as np
2
3 SIZE = 100
4
5 mat = np.random.rand(SIZE, SIZE)
6 double_column = 2 * mat[:, 0]
7 double_row = 2 * mat[0, :]
```

which is a slightly modified version of the code from the book - using `rand` instead of `randint` and floating point numbers instead of integers.

1. Make a Python program that measures the execution time of `2 * mat[:, 0]` and `2 * mat[0, :]`. Measure the time for at least 1000 repetitions. Hint: remember what you did in Week 2, Exercise 2.5.

Answer:

```
1 from time import perf_counter as time
2
3 import numpy as np
4
5 SIZE = 100
6
7 n_repeat = int(1e3)
8 mat = np.random.rand(SIZE, SIZE)
9
10 trow = time()
11 for _ in range(n_repeat):
12     mat[0, :] * 1.01
13 trow = time() - trow
14
15 tcol = time()
16 for _ in range(n_repeat):
17     mat[:, 0] * 1.01
18 tcol = time() - tcol
19
```

```
20 print('trow =', trow / n_repeat)
21 print('tcol =', tcol / n_repeat)
```

2. **Autolab** Make a job script that runs your program. Submit it to the hpc queue, use a node with an Intel Xeon Gold 6126, Intel Xeon Gold 6142 or Intel Xeon Gold 6226R processor, and request a single core. Hint: remember the exercises from week 1.
-

Answer: Autolab exercise - answer omitted

3. Modify your Python program and/or your jobscript to measure the time for SIZE ranging from 10^1 to $10^{4.5}$. Do the measurement from a batch job - this is important, so you have control over the hardware!
Hint: Use `np.logspace` to get logarithmically spaced numbers for SIZE.
-

Answer:

```
1 from time import perf_counter as time
2
3 import numpy as np
4
5 # Rows vs. Columns
6 # ns = [100, 1000, 10000] # <-- From the book
7 ns = np.round(np.logspace(1, 4.5, 30))
8 trows = []
9 tcols = []
10 n_repeat = int(1e3)
11
12 for n in ns:
13     n = int(n)
14     mat = np.random.rand(n, n)
15
16     trow = time()
17     for _ in range(n_repeat):
18         mat[0, :] * 1.01
19     trow = time() - trow
20
21     tcol = time()
22     for _ in range(n_repeat):
23         mat[:, 0] * 1.01
24     tcol = time() - tcol
25
26     trows.append(trow / n_repeat)
```

```

27     tcols.append(tcol / n_repeat)
28
29 print(mat.dtype)
30 print('ns =', list(ns))
31 print('trows =', trows)
32 print('tcols =', tcols)

```

4. Make a loglog plot of the performance of the row and column doubling as MFLOP/s over the size of the matrix in kilobytes. Do they perform the same? How does their performance align with the sizes of the CPU caches? Hint: you can read the cache sizes from the `lscpu` output.

Answer: I used an Intel Xeon Gold 6126 processor. The output of `lscpu` should be:

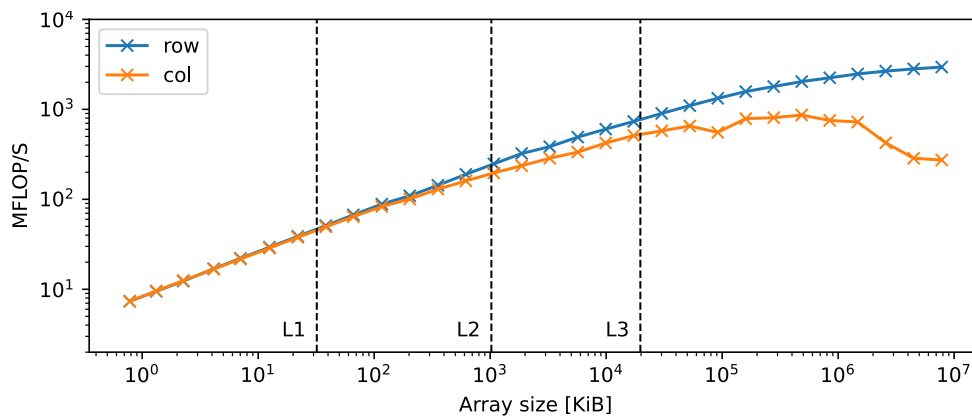
```

1 Architecture:          x86_64
2 CPU op-mode(s):        32-bit, 64-bit
3 Byte Order:             Little Endian
4 CPU(s):                 24
5 On-line CPU(s) list:   0-23
6 Thread(s) per core:    1
7 Core(s) per socket:    12
8 Socket(s):              2
9 NUMA node(s):          2
10 Vendor ID:             GenuineIntel
11 CPU family:             6
12 Model:                 85
13 Model name:            Intel(R) Xeon(R) Gold 6126 CPU @ 2.60GHz
14 Stepping:              4
15 CPU MHz:               999.960
16 BogoMIPS:              5200.00
17 Virtualization:        VT-x
18 L1d cache:             32K
19 L1i cache:             32K
20 L2 cache:              1024K
21 L3 cache:              19712K
22 NUMA node0 CPU(s):     0-11
23 NUMA node1 CPU(s):     12-23
24 Flags:                 fpu vme de pse tsc msr <rest omitted...>

```

The size of the L1 data cache is under `L1d cache:` and is 32 kilobytes (KB). The L2 is under `L2 cache:` and is 1024 kilobytes (KB) or 1 megabyte (MB). Finally, the L3 cache is under `L3 cache:` and is 19712 kilobytes (KB) or 19.25 megabytes (MB).

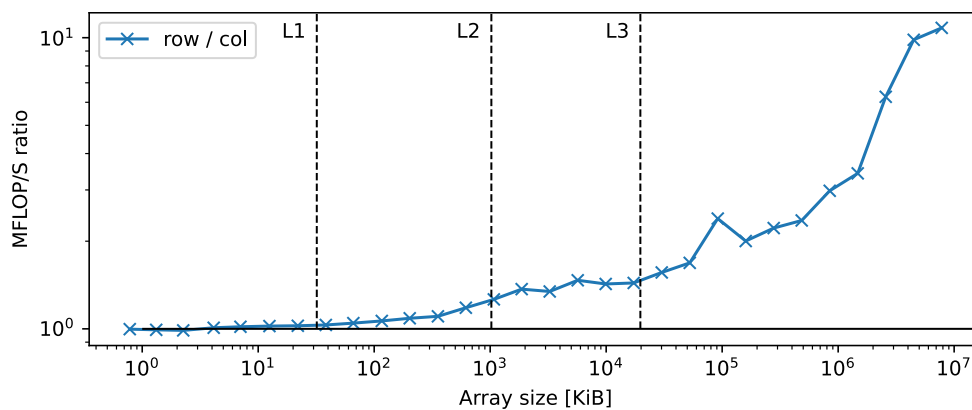
The performance plot is:



When the array is small enough to fit in the L1 cache, both scaling methods perform similarly. As the array grows to fit only in the L2 cache, their performance diverges. The reason is that column scaling now incurs cache misses because it accesses almost the entire array, whereas row scaling remains efficient because the row still fits in the L1 cache. As the array size increases, column scaling suffers more from cache misses, including costlier L2 and L3 misses, which makes the performance gap even wider.

a) To make the performance difference clearer, plot the ratio of MFLOPS/s.

Answer: The ratio of the performances is:



5. Let us focus on the row scaling. Modify the code so `mat` is a row vector, i.e., `mat = np.random.rand(1, SIZE)`. Measure the time for `SIZE` ranging from 10^2 to 10^8 using at least 100 repetitions. Again, do the measurement from a batch job.
Hint: Again, use `np.logspace` to generate values for `SIZE`.

Answer:

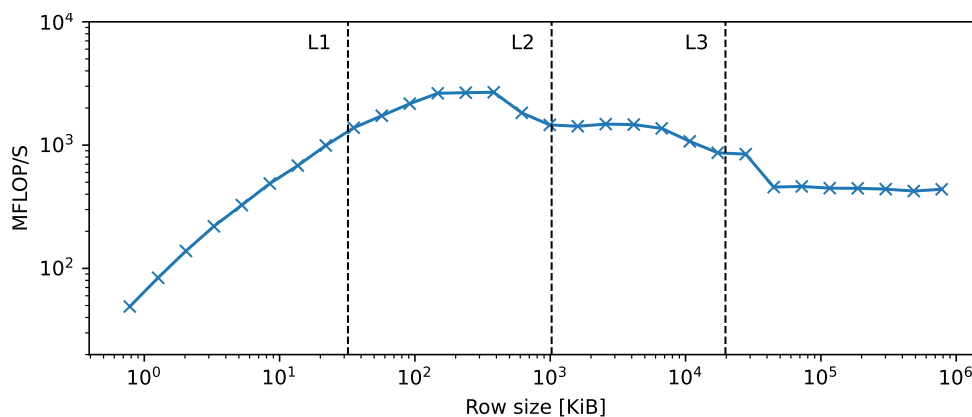
```

1  from time import perf_counter as time
2
3  import numpy as np
4
5  # Only row
6  ns = np.ceil(np.logspace(2, 8, 30))
7  trows = []
8  n_repeat = int(1e2)
9  for n in ns:
10     n = int(n)
11     mat = np.random.rand(1, n)
12     trow = time()
13     for _ in range(n_repeat):
14         mat[0, :] * 2
15     trow = time() - trow
16     trows.append(trow / n_repeat)
17
18  print('ns =', list(ns))
19  print('trows =', trows)

```

6. **Autolab** Make a loglog plot of the performance of the row doubling as MFLOP/s over the size of the row vector in kilobytes. What do you see? Do the performance changes align with the cache sizes?

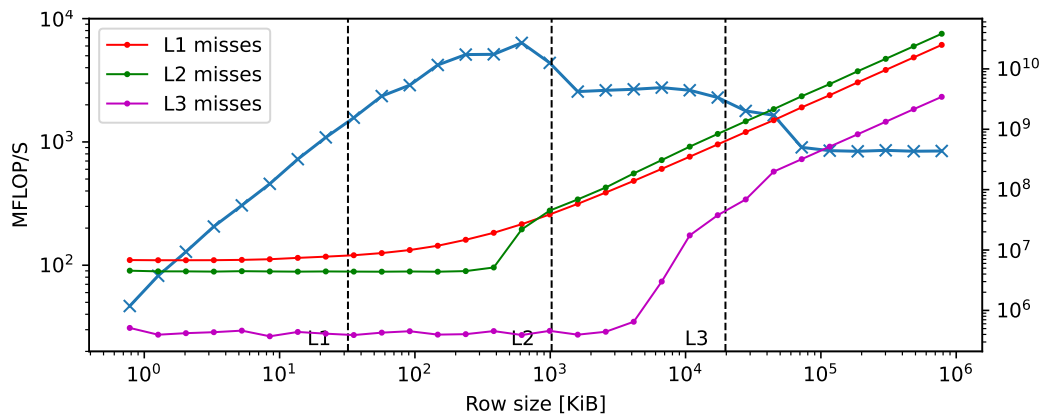
Answer: The performance plot is:



The increase in performance while the row still fits in the L1 is likely due to the Python's overhead dominating the time measurements. This is the price we pay for using Python. Otherwise, we do observe that performance decreases in stair step fashion around cache boundaries, as expected.

Optional details: We can verify that what we're seeing is indeed due to cache effects by mea-

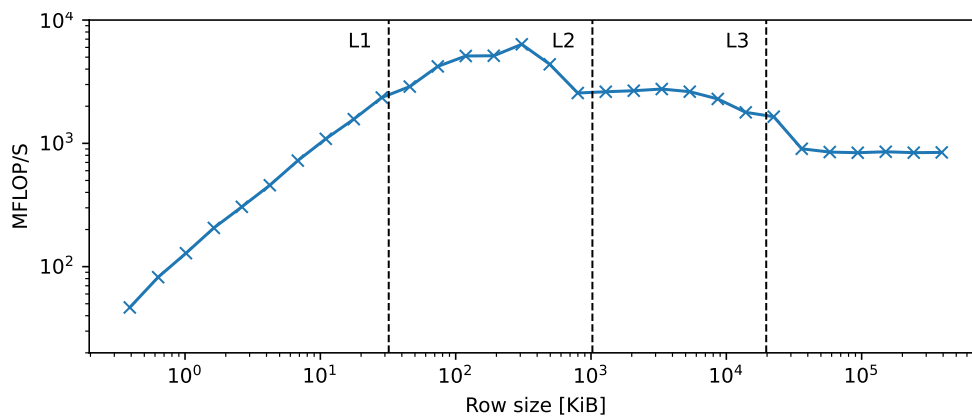
asuring cache misses with per f. In the plot below, we overlay the number of cache misses as a function of the array size:



The uptick in cache misses corresponds well with the dips in performance. Note, the use of per f is *not* part of 02613 and you are not expected to do this! If you find this interesting, see [02614 High Performance Computing](#).

- a) (Optional:) Redo the experiment using `mat = mat.astype('float32')`. Do you observe the same pattern?

Answer: The performance plot is:



The pattern is more or less the same, which is expected.

2. Efficient data storage with Blosc

The goal of this exercise is to analyze when the time required to read the compressed data and to uncompress it is smaller than the time to read the raw data. The exercise is based on the example from section 6.2.1 in Fast Python. We are going to use Blosc to perform efficient compression and decompression algorithms. Use the following supporting functions:

```

1 import os
2 import blosc
3 import numpy as np
4
5
6 def write_numpy(arr, file_name):
7     np.save(f"{file_name}.npy", arr)
8     os.sync()
9
10
11 def write_blosc(arr, file_name, cname="lz4"):
12     b_arr = blosc.pack_array(arr, cname=cname)
13     with open(f"{file_name}.bl", "wb") as w:
14         w.write(b_arr)
15     os.sync()
16
17
18 def read_numpy(file_name):
19     return np.load(f"{file_name}.npy")
20
21
22 def read_blosc(file_name):
23     with open(f"{file_name}.bl", "rb") as r:
24         b_arr = r.read()
25     return blosc.unpack_array(b_arr)

```

We add `os.sync()` to force the disk to flush, cleaning the operating system IO buffers as much as possible to ensure a fair comparison between the methods (see Section 6.2 in Fast Python).

1. **Autolab** Write a Python program that takes a number n as a command line argument. It must then generate a 3 dimensional NumPy array of zeros with size $n \times n \times n$ and `dtype='uint8'`. The program must then measure and print the time it takes to perform each of the following operations:
 - Save the array to a file using the provided `write_numpy`.
 - Save the array to a file using the provided `write_blosc`.
 - Read the array from the created file using the provided `read_numpy`.
 - Read the array from the created file using the provided `read_blosc`.

Note, you must print the time for each operation separately, i.e., four time values should be printed. You may print them on a single line separated by a space or on 4 separate lines.

Answer: Autolab exercise - answer omitted

2. **Autolab** Make a job script that runs your program with $n = 256, 512$ and 1024 and submit it to the hpc queue. Request 1 core and remember to request enough memory to keep the arrays in memory.

Answer: Autolab exercise - answer omitted

3. Do the same, but now, instead of zero entries, the array will have tiled values of integers up to 256. Hint: You may use the following snippet to generate the array:

```
1 tiled_array = np.tile(  
2     np.arange(256, dtype='uint8'),  
3     (n // 256) * n * n,  
4 ).reshape(n, n, n)
```

Answer: See hint.

4. Do it one last time, but now using random integer entries from 0 to 256. Hint: Use the `np.random.randint` function.

Answer: Generate the array with

```
1 rand_array = np.random.randint(  
2     0, 256, size=(n,) * 3, dtype='uint8'  
3 )
```

-
5. **Autolab** Compare the time it takes to write and read the files in the three items above for the different n . When should we use `blosc`? Why? Compare also the sizes the generated files.

Answer: Autolab exercise - answer omitted

-
- a) In the blosc implementation, change the compression algorithm from `cname="lz4"` to `cname="zstd"`. What improvements do you observe? What was the cost of these improvements?
-

Answer: Autolab exercise - answer omitted
